

Chapter 1: Introduction to Python

What Is Python?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its name is a nod to the British comedy group Monty Python. Today Python is one of the most widely used languages in the world, ranking in the top 3 of TIOBE, PYPL, and the Stack Overflow Developer Survey every year.

Design Philosophy

Python's guiding principles are documented in PEP 20 (The Zen of Python). Core ideas include: Beautiful is better than ugly. Explicit is better than implicit. Simple is better than complex. Readability counts. There should be one obvious way to do it. These principles explain why Python code looks clean and readable compared to other languages.

A Brief History

Guido van Rossum began designing Python in the late 1980s at CWI in the Netherlands as a successor to the ABC language. Python 1.0 was released in January 1994. Python 2.0 added list comprehensions, garbage collection, and Unicode support in 2000. Python 3.0, released in December 2008, was a backward-incompatible revision that cleaned up years of technical debt. Python 2 reached its official end-of-life in January 2020. The latest stable release is Python 3.12.

Key Features

Dynamically typed: you do not declare variable types. Garbage-collected: memory is managed automatically. Multi-paradigm: supports procedural, object-oriented, and functional programming. Batteries included: Python ships with a massive standard library plus access to 450,000 third-party packages on PyPI.

Where Python Is Used

Data Science and Machine Learning: NumPy, Pandas, Scikit-Learn, TensorFlow, PyTorch have made Python the dominant language for AI research and production ML. Web Development: Django and Flask power millions of websites worldwide. Scripting and Automation: Python replaces manual repetitive tasks in DevOps, testing, and system administration. Scientific Computing: SciPy, Matplotlib, and Jupyter Notebooks are standard tools in academic research. Cybersecurity: penetration testers write exploit scripts and analysis tools in Python.

Companies Using Python

Google, Instagram, Spotify, Dropbox, Netflix, NASA, CERN, Reddit, and YouTube all use Python extensively. Google maintains CPython (the reference implementation) and contributes regularly to the language.

Getting Started

Download Python from python.org. The REPL (Read-Eval-Print Loop) lets you type statements interactively and see results immediately. Run: `print('Hello, Python!')` -- that is a complete Python program. No semicolons, no curly braces, no class declarations required. That simplicity is Python's hallmark.

Why Learn Python First?

Python is the recommended first language because its syntax is forgiving and readable, the feedback loop is instant, and the same language scales all the way to complex production systems. Whether you want to build websites, analyze data, train machine learning models, or automate boring tasks, Python is the right tool for the job.

Chapter 2: Python Basics - Variables, Types, and Operators

Variables and Assignment

A variable is a name that points to an object in memory. Create one with assignment: `x = 42`. No type declaration is needed -- Python infers the type automatically. Python is dynamically typed: one variable can hold an integer, then be reassigned to a string on the next line. Variable names are case-sensitive (`age` and `Age` are different) and use `snake_case` by convention (`total_price`, `user_name`).

Core Data Types

`int` -- Whole numbers of arbitrary size. Python 3 integers never overflow: `2**100` computes exactly.

`float` -- IEEE 754 double-precision. Be aware: `0.1 + 0.2` equals `0.30000000000000004` due to binary representation.

`str` -- Immutable sequences of Unicode code points. Defined with single, double, or triple quotes.

`bool` -- `True` and `False` are the only two boolean values. `bool` is a subclass of `int`.

`NoneType` -- The `None` singleton represents the absence of a value, similar to `null` in other languages.

Type Checking and Conversion

Use `type()` to inspect a variable: `type(3.14)` returns `float`. Convert with `int()`, `float()`, `str()`, `bool()`. Python requires explicit conversion -- no hidden coercion -- which prevents surprising bugs.

Strings in Depth

Escape sequences: `\n` is newline, `\t` is tab, `\\` is a literal backslash. Raw strings (`r'path'`) suppress escape processing -- useful for file paths and regular expressions. F-strings (Python 3.6+) embed expressions: `name = 'Alice'; print(f'Hello, {name}!')` -- this is the modern preferred format method.

Arithmetic Operators

`+` addition, `-` subtraction, `*` multiplication, `/` true division (always float), `//` floor division, `%` modulus (remainder), `**` exponentiation. Augmented assignment: `+=`, `-=`, `*=`, `/=`, `//=`, `**=`, `%=`.

Comparison and Logical Operators

Comparison: `==` (equal), `!=` (not equal), `<` (less than), `>` (greater than), `<=`, `>=`. All return `bool`. Logical: `and` returns `True` if both sides are `True`. or returns `True` if at least one is `True`. `not` negates. Chained comparisons work: `0 < x < 100` is valid and idiomatic Python.

The `print()` and `input()` Functions

`print()` writes to `stdout` with an automatic newline. Customize: `print('a', 'b', sep='-', end='!')`. `input()` reads a string from the user. Always convert if you need a number: `age = int(input('Age: '))`.

Comments and Docstrings

Single-line comments start with `#`. Triple-quoted strings serve as docstrings for functions, classes, and modules. Docstrings are accessible via the built-in `help()` function.

Chapter 4: Advanced Python - OOP, Modules, Generators, and Decorators

Object-Oriented Programming (OOP)

Every value in Python is an object. The class keyword defines a blueprint. `__init__` is the constructor invoked automatically when creating an instance. `self` is the first parameter of every instance method -- it references the current object.

Inheritance

A child class inherits attributes and methods from its parent: `class Dog(Animal)`. `super()` calls the parent's implementation. Python supports multiple inheritance. The MRO (Method Resolution Order) uses the C3 linearization algorithm to determine which parent method is called when names conflict.

Dunder (Magic) Methods

`__repr__` and `__str__` control string representation. `__len__` enables `len(obj)`. `__add__` enables the `+` operator. `__iter__` and `__next__` make an object iterable. `__enter__` and `__exit__` enable the with statement (context managers).

Properties and Encapsulation

`@property` turns a method into a read-only attribute. `@attribute.setter` adds a setter. Double-underscore prefix (`__x`) triggers name mangling: becomes `_ClassName__x`, making the attribute class-private. Single underscore (`_x`) means internal use by convention.

Generators and Iterators

An iterator implements `__iter__` and `__next__`. Generators use `yield` to produce values one at a time: `def countdown(n): while n > 0: yield n; n -= 1`. Generator expressions are lazily evaluated: `(x**2 for x in range(1000000))` consumes almost no memory.

Decorators

A decorator wraps a function to modify its behavior without changing its source code. `@timer` before a function definition is sugar for `func = timer(func)`. Flask uses `@app.route()`. Django uses `@login_required`. `functools.wraps` preserves metadata.

Context Managers

The with statement guarantees cleanup even if exceptions occur. `with open('file.txt') as f:` automatically calls `f.close()`. Custom context managers use `__enter__`/`__exit__` or `@contextmanager` from `contextlib`.

Modules and Packages

A module is a `.py` file. `import math; math.sqrt(16)`. `from datetime import datetime`. Packages are directories with `__init__.py`. Virtual environments (`python -m venv env`) isolate dependencies per project. `pip install name` fetches from PyPI.

Type Hints

Python 3.5+ supports optional type annotations: `def add(a: int, b: int) -> int`. Not enforced at runtime but used by static checkers (mypy) and IDEs for autocomplete.

Chapter 5: Expert Python - Concurrency, Performance, Testing, and Ecosystem

Concurrency Models

Python has three main concurrency approaches. Threading (threading module): OS threads share memory. The Global Interpreter Lock (GIL) prevents true CPU parallelism in CPython -- best for I/O-bound tasks (network requests, file reads). Multiprocessing: separate processes with own interpreter and memory, bypassing the GIL -- ideal for CPU-bound work (image processing, numerical computation). Asyncio: a single-threaded event loop using `async/await` for cooperative multitasking -- best for high-concurrency I/O like web servers handling thousands of connections simultaneously.

async / await

`async def` defines a coroutine. `await` suspends it until the target completes. `asyncio.run()` is the entry point. `aiohttp` provides async HTTP. FastAPI is an async REST framework achieving very high throughput.

Performance Optimization

Always profile before optimizing: `python -m cProfile my_script.py`. NumPy operations in C are orders of magnitude faster than Python loops. Cython compiles annotated Python to C. Numba JIT-compiles numerical functions at runtime. PyPy is an alternative interpreter with JIT compilation. `functools.lru_cache` memoizes expensive function calls. For string building, use `".join(parts)"` not `+` concatenation.

Memory Management

Python uses reference counting plus a cyclic garbage collector. `del` removes a name binding; the object is freed when its reference count reaches zero. `sys.getsizeof()` measures object size. `tracemalloc` profiles memory allocation. Use NumPy typed arrays instead of Python lists for large numeric data.

Testing

`pytest` is the industry standard. Test functions are prefixed `test_`. `pytest` auto-discovers them. Fixtures provide reusable setup/teardown via `@pytest.fixture`. `Parametrize` runs one test with many inputs. `coverage.py` measures what percentage of code is covered. TDD: write tests before implementation.

Logging and Debugging

Use the logging module instead of `print` in production. Configure levels: `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`. Direct output to files, rotate logs, format as JSON for log aggregation systems. `pdb` is the built-in interactive debugger. Python 3.7+: use `breakpoint()` as a portable entry point. Static analysis: `pylint`, `flake8` check style; `mypy` checks types.

Packaging and Distribution

Modern Python packages use `pyproject.toml` (PEP 517/518) as build configuration. Tools: `Poetry`, `Hatch`, `setuptools`. Publish to PyPI with `twine upload dist/*`. Follow semantic versioning: `MAJOR.MINOR.PATCH`.

The Python Ecosystem

Web: Django (batteries-included), Flask (micro), FastAPI (async, auto-docs). Data: NumPy, Pandas, Polars (Rust-backed DataFrames). ML: Scikit-Learn, XGBoost, LightGBM. Deep Learning: PyTorch (research), TensorFlow/Keras (production). Visualization: Matplotlib, Seaborn, Plotly. ORMs: SQLAlchemy, Django ORM. CLI: Click, Typer. Config/Validation: Pydantic.

Pythonic Code Principles

List comprehensions over `map/filter`. `enumerate()` over manual index counters. Tuple unpacking over indexing. Context managers for resources. Duck typing over `isinstance()` checks. The goal is always readability, explicitness, and simplicity -- the true Python way.